

Post_Process.py

```
1  # Post_Process.py
2      # Takes [var], [Misc], "APDL_Eigen_Internal.txt", "APDL_Nonlin_Internal" as input
3      # 6 Functions for calculating:
4          # 1. Local Buckling                [LB]
5          # 2. Normal Force                  [NF]
6          # 3. Shear                        [S]
7          # 4. Torsion                      [T]
8          # 5. Bending, Normal and Shear    [BNS]
9          # 6. Buckling Resistance          [BR]
10         # 7. Interaction                   [IN]
11         # 8. Brace-Step                   [BS]
12     # Outputs this as a .txt file for evaluation
13     # Output highest utilization factor as list
14     #   FORMAT:
15     #   [Column NF, Brace NF]
16
17     # Import tools
18     from pathlib import Path
19     import pandas as pd
20     import numpy as np
21     import re
22     import math
23     from io import StringIO
24     import SW_Import as SW
25
26     # Function to calculate the span of horizontal brace - It works, but i (Christian) don't understand this
27     def _brace_span_mm(var, misc, tol=1e-3):
28         co = SW.import_SW(str(Path("IGS") / misc["SW_filename"]))
29
30         # If a target radius is provided, move the column-connected points
31         # radially in the x-z plane while keeping y unchanged
32         rad = var.get("rad")
33         if isinstance(rad, dict):
34             rad = rad.get("value")
35         if rad is not None:
36             # Find vertical column footprints in the x-z plane
37             col = {
38                 (x1, z1)
39                 for x1, y1, z1, x2, y2, z2 in co
40                 if abs(x1 - x2) < tol and abs(z1 - z2) < tol
41             }
42
43             # Map each footprint to the same direction but new radius = rad
44             ln = {}
45             for x, z in col:
46                 h = math.hypot(x, z)
47                 if h < tol:
48                     ln[(x, z)] = (x, z) # avoid dividing by ~0
49                 else:
50                     ln[(x, z)] = (x * rad / h, z * rad / h)
51
52             def adj(x, y, z):
53                 if (x, z) in ln:
54                     x_new, z_new = ln[(x, z)]
55                     return (x_new, y, z_new)
56                 return (x, y, z)
57
58             co = [
59                 (*adj(x1, y1, z1), *adj(x2, y2, z2))
60                 for x1, y1, z1, x2, y2, z2 in co
61             ]
62
```

Post_Process.py

```
63     # Find the first member that is horizontal:
64     #   - same y-coordinate
65     #   - but different x/z coordinates
66     for x1, y1, z1, x2, y2, z2 in co:
67         is_horizontal = abs(y2 - y1) < tol
68         has_horizontal_span = abs(x2 - x1) >= tol or abs(z2 - z1) >= tol
69
70         if is_horizontal and has_horizontal_span:
71             return math.sqrt((x2 - x1)**2 + (y2 - y1)**2 + (z2 - z1)**2)
72
73 class PostProcessor:
74
75     def __init__(self, var, Misc, opti_settings):
76         # Store inputs
77         self.var = var
78         self.Misc = Misc
79         self.opti_settings = opti_settings
80
81         self.n = int(opti_settings.get("n_mast_segments"))
82         self.mast_height = float(opti_settings.get("mast_segment_height"))
83         self.multi_size_columns = bool(opti_settings.get("multi_size_columns"))
84         self.multi_size_braces = bool(opti_settings.get("multi_size_braces"))
85         self.brace_split = bool(opti_settings.get("brace_split"))
86         self.E_mod = Misc["E_mod"]
87         self.f_y = Misc["f_y"]
88         self.f_y_brace = Misc["f_y_brace"]
89
90         # Build per-segment section radii for columns and braces
91         self.column_R0, self.column_R1 = self._build_section_radii("d0", "t0", self.multi_size_columns)
92         self.column_d0_t0 = self._build_section_dimensions("d0", "t0", self.multi_size_columns)
93
94         # Handle braces based on split setting
95         if self.brace_split:
96             # Horizontal braces
97             self.brace_horiz_R2, self.brace_horiz_R3 = self._build_section_radii("d1_h", "t1_h", self.multi_s
98             # Cross braces
99             self.brace_cross_R2, self.brace_cross_R3 = self._build_section_radii("d1_c", "t1_c", self.multi_s
100
101             # Build per-segment diameter/thickness data for class checks
102             self.brace_horiz_d1_t1 = self._build_section_dimensions("d1_h", "t1_h", self.multi_size_braces)
103             self.brace_cross_d1_t1 = self._build_section_dimensions("d1_c", "t1_c", self.multi_size_braces)
104             self.brace_d1_t1 = self.brace_horiz_d1_t1 + self.brace_cross_d1_t1
105         else:
106             # Combined braces (same size for both)
107             self.brace_R2, self.brace_R3 = self._build_section_radii("d1", "t1", self.multi_size_braces)
108             self.brace_d1_t1 = self._build_section_dimensions("d1", "t1", self.multi_size_braces)
109
110         # Load nonlinear results and separate columns from braces
111         self.df_nonlin = self.read_forces("Ansout/APDL_Nonlin_Internal.txt")
112         self.df_col = self.df_nonlin[self.df_nonlin["Member"].str.startswith("ColMember")].copy()
113
114         if self.brace_split:
115             self.df_horiz = self.df_nonlin[self.df_nonlin["Member"].str.startswith("HorizMember")].copy()
116             self.df_cross = self.df_nonlin[self.df_nonlin["Member"].str.startswith("CrossMember")].copy()
117         else:
118             # If not split, combine all braces
119             self.df_brace = self.df_nonlin[
120                 (self.df_nonlin["Member"].str.startswith("HorizMember")) |
121                 (self.df_nonlin["Member"].str.startswith("CrossMember"))
122             ].copy()
```

Post_Process.py

```
123     self._annotate_member_sections()
124     self.df_brace = self._combined_brace_dataframe()
125
126     # Extract numeric value from the var dictionary, supporting nested dict shape
127     def _get_value(self, key):
128         value = self.var.get(key)
129         if isinstance(value, dict):
130             return float(value["value"])
131         return float(value)
132
133     # Build inner and outer radii for one or multiple section sizes
134     def _build_section_radii(self, d_key, t_key, multi_size):
135         if multi_size:
136             inner = []
137             outer = []
138             for i in range(1, self.n + 1):
139                 d = self._get_value(f"{d_key}_{i}")
140                 t = self._get_value(f"{t_key}_{i}")
141                 outer.append(d / 2.0)
142                 inner.append(d / 2.0 - t)
143             return inner, outer
144
145         d = self._get_value(d_key)
146         t = self._get_value(t_key)
147         return [d / 2.0 - t], [d / 2.0]
148
149     # Build diameter/thickness tuples for class checks and brace sizing
150     def _build_section_dimensions(self, d_key, t_key, multi_size):
151         if multi_size:
152             return [
153                 (self._get_value(f"{d_key}_{i}"), self._get_value(f"{t_key}_{i}"))
154                 for i in range(1, self.n + 1)
155             ]
156         return [(self._get_value(d_key), self._get_value(t_key))]
157
158     # Convert Y location to mast segment indices for section lookup
159     def _segment_series(self, y_series):
160         segments = np.ceil(y_series / self.mast_height).astype(int)
161         segments = np.maximum(segments, 1)
162         segments = np.minimum(segments, self.n)
163         return segments
164
165     # Assign section radii to each member row based on its segment position
166     def _annotate_member_sections(self):
167         if not self.df_col.empty:
168             self.df_col["segment"] = self._segment_series(self.df_col["Y_LOC"])
169             if len(self.column_R0) > 1:
170                 self.df_col["R0"] = self.df_col["segment"].map(lambda s: self.column_R0[s - 1])
171                 self.df_col["R1"] = self.df_col["segment"].map(lambda s: self.column_R1[s - 1])
172             else:
173                 self.df_col["R0"] = self.column_R0[0]
174                 self.df_col["R1"] = self.column_R1[0]
175
176         if self.brace_split:
177             # Handle separate horizontal and cross braces
178             if not self.df_horiz.empty:
179                 self.df_horiz["segment"] = self._segment_series(self.df_horiz["Y_LOC"])
180                 if len(self.brace_horiz_R2) > 1:
181                     self.df_horiz["R2"] = self.df_horiz["segment"].map(lambda s: self.brace_horiz_R2[s - 1])
182                     self.df_horiz["R3"] = self.df_horiz["segment"].map(lambda s: self.brace_horiz_R3[s - 1])
183                 else:
184                     self.df_horiz["R2"] = self.brace_horiz_R2[0]
```

Post_Process.py

```
185         self.df_horiz["R3"] = self.brace_horiz_R3[0]
186
187     if not self.df_cross.empty:
188         self.df_cross["segment"] = self._segment_series(self.df_cross["Y_LOC"])
189         if len(self.brace_cross_R2) > 1:
190             self.df_cross["R2"] = self.df_cross["segment"].map(lambda s: self.brace_cross_R2[s - 1])
191             self.df_cross["R3"] = self.df_cross["segment"].map(lambda s: self.brace_cross_R3[s - 1])
192         else:
193             self.df_cross["R2"] = self.brace_cross_R2[0]
194             self.df_cross["R3"] = self.brace_cross_R3[0]
195     else:
196         # Handle combined braces
197         if not self.df_brace.empty:
198             self.df_brace["segment"] = self._segment_series(self.df_brace["Y_LOC"])
199             if len(self.brace_R2) > 1:
200                 self.df_brace["R2"] = self.df_brace["segment"].map(lambda s: self.brace_R2[s - 1])
201                 self.df_brace["R3"] = self.df_brace["segment"].map(lambda s: self.brace_R3[s - 1])
202             else:
203                 self.df_brace["R2"] = self.brace_R2[0]
204                 self.df_brace["R3"] = self.brace_R3[0]
205
206     def _combined_brace_dataframe(self):
207         if self.brace_split:
208             frames = [df for df in (self.df_horiz, self.df_cross) if not df.empty]
209             if not frames:
210                 return pd.DataFrame()
211             # Combine horizontal and cross braces into one continuous DataFrame.
212             # Use a fresh index to avoid separate original index spaces.
213
214             return pd.concat(frames, ignore_index=True)
215         return self.df_brace
216
217     def _brace_utilization(self, func):
218         if self.brace_split:
219             values = [
220                 func(df, "R3", "R2", self.f_y_brace)
221                 for df in (self.df_horiz, self.df_cross)
222                 if not df.empty
223             ]
224             if values:
225                 return pd.concat(values, ignore_index=False)
226             return pd.Series(dtype=float)
227         return func(self.df_brace, "R3", "R2", self.f_y_brace)
228
229     # Function to read and parse forces
230     # NOTE: I have used Copilot for most of this function, so understanding is low ...
231     def read_forces(self, filepath: str) -> pd.DataFrame:
232         blocks = []
233         current_member = None
234         colnames = None
235         buffer = []
236
237         with open(filepath, "r") as f:
238             lines = f.readlines()
239
240         buffer = []
241
242         for line in lines:
243             # Detect new section header
244             if line.startswith("NS"):
245                 if buffer and colnames and current_member:
246                     df = pd.read_csv(
```

Post_Process.py

```
247         StringIO("".join(buffer)),
248         sep=r"\s+",
249         engine="python",
250         names=colnames
251     )
252     if "Y_LOC" in df.columns:
253         df = df[df["Y_LOC"] <= 4080]
254
255     df["Member"] = current_member
256     blocks.append(df)
257
258     buffer = []
259     current_member = line.strip().replace("NS ", "").replace(".", "")
260     continue
261
262     if "ElemID" in line and "NF" in line:
263         tokens = line.split()
264         colnames = [tok for tok in tokens if "[" not in tok]
265         continue
266
267     if re.match(r"^\s*\d", line):
268         buffer.append(line)
269
270     if buffer and colnames and current_member:
271         df = pd.read_csv(
272             StringIO("".join(buffer)),
273             sep=r"\s+",
274             engine="python",
275             names=colnames
276         )
277         if "Y_LOC" in df.columns:
278             df = df[df["Y_LOC"] <= 4078]
279         df["Member"] = current_member
280         blocks.append(df)
281
282     return pd.concat(blocks, ignore_index=True)
283
284 # Function to evaluate all constraints at once
285 def Util_list(self):
286
287     util_data = {
288         "Util_LB": self.Util_LB(),
289         "Util_NF": self.Util_NF(),
290         "Util_S": self.Util_S(),
291         "Util_T": self.Util_T(),
292         #"Util_BNS": self.Util_BNS(),
293         "Util_BR": self.Util_BR(),
294         "Util_IN": self.Util_IN(),
295         "Util_BS_sig": self.Util_BS()[0], # stress
296         "Util_BS_defl": self.Util_BS()[1], # deflection
297     }
298
299     return util_data
300
301 # Local Buckling [Timeshenko p. 458]
302 def Util_LB(self):
303     # Import E_mod
304     E_mod = self.E_mod
305
306     # Poissons Ratio
307     v = 0.3
308
```

Post_Process.py

```
309     def LocalBucklingFun(df_member, Ro_col, Ri_col):
310         df = df_member.copy()
311
312         P_cr = (2 * math.pi * E_mod * (df[Ro_col] - df[Ri_col])**2) / math.sqrt(3 * (1 - v**2))
313
314         N_raw = df["NF"].to_numpy(dtype=float)
315         N_comp = np.maximum(-N_raw, 0.0) # only compression counts
316
317         return pd.Series(N_comp / P_cr, index=df.index, name="Util_LB")
318
319     Util_LB_col = LocalBucklingFun(self.df_col, "R1", "R0")
320     Util_LB_brace = LocalBucklingFun(self.df_brace, "R3", "R2")
321
322     return Util_LB_col, Util_LB_brace
323
324 # Normal Force [6.2.4]
325 def Util_NF(self):
326
327     # Import f_y
328     f_y = self.f_y
329
330     def NormalForceFun(df_member, Ro_col, Ri_col, f_y):
331         # Area
332         A = np.pi * ((df_member[Ro_col]**2) - (df_member[Ri_col]**2))
333
334         # Compression/Tension Utilization (1993-1-1 [6.2.3] and [6.2.4])
335         df_member["Util_NF"] = df_member["NF"].abs() / (A * f_y)
336
337         # Utilization Ratio [1993-1-1 [6.2.4] - p. 49 (6.9)]
338         return df_member["Util_NF"]
339
340     Util_NF_col = NormalForceFun(self.df_col, "R1", "R0", f_y) # Column
341     Util_NF_brace = self._brace_utilization(NormalForceFun) # Brace
342
343     return Util_NF_col, Util_NF_brace
344
345 # Shear Force [6.2.6]
346 def Util_S(self):
347
348     # Import Misc
349     f_y = self.f_y
350
351     def shearFun(df_member, Ro_col, Ri_col, f_y):
352         A = np.pi * ((df_member[Ro_col]**2) - (df_member[Ri_col]**2))
353
354         # Av Area [6.2.6 (3) with option (g)]
355         Av = 2/math.pi * A
356
357         # Design Plastic Shear Resistance [6.2.6 (6.18)]
358         V_cRd = Av * (f_y / math.sqrt(3))
359
360         # Utilization Ratio [6.2.6 (6.17)]
361         # Calculate both directions separately so all shear utils are available.
362         df_member["Util_Sy"] = df_member["Vy"].abs() / V_cRd
363         df_member["Util_Sz"] = df_member["Vz"].abs() / V_cRd
364
365         return df_member[["Util_Sy", "Util_Sz"]]
366
367     Util_S_col = shearFun(self.df_col, "R1", "R0", f_y) # Column
368     Util_S_brace = self._brace_utilization(shearFun) # Brace
369
370     return Util_S_col, Util_S_brace
```

```

371
372 # Torsion [6.2.7]
373 def Util_T(self):
374
375     # Import Misc
376     f_y = self.f_y
377
378     def torsionFun(df_member, Ro, Ri, f_y):
379         D0 = df_member[Ro] * 2 # Outer Diameter
380         Di = df_member[Ri] * 2 # Inner Diameter
381
382         # Torsion Design Resitance [6.2.7 (6.23)]
383         T_Rd = (math.pi/16 * (D0**4 - Di**4) / D0) * f_y / math.sqrt(3)
384
385         # Torsion Check [6.2.7 (6.23)]
386         df_member["Util_T"] = df_member["T"].abs() / T_Rd
387
388         # Return Utilization Ratio
389         return df_member["Util_T"]
390
391     Util_T_col = torsionFun(self.df_col, "R1", "R0", f_y) # Column
392     Util_T_brace = self._brace_utilization(torsionFun) # Brace
393
394     return Util_T_col, Util_T_brace
395
396 # Bending, Normal and Shear [6.2.9]
397 def Util_BNS(self):
398     # Import f_y
399     f_y = self.f_y
400     f_y_brace = self.f_y_brace
401
402     def bnsFun(df_member, Ro, Ri, f_y):
403         D0 = df_member[Ro] * 2
404         Di = df_member[Ri] * 2
405         t = (D0 - Di) / 2
406
407         # Area
408         A = np.pi * ((df_member[Ro]**2) - (df_member[Ri]**2))
409         Aw = (A - 2 * D0 * t) / A
410         Aw = np.clip(Aw, 0, 0.5)
411
412         # Design Moment Resitance
413         M_Rd = (D0**3 - Di**3) / 6 * f_y
414
415         # Design Normal Force Resistance
416         N_Rd = A * f_y
417
418         # Forces (Here read as values)
419         N = df_member["NF"].abs()
420         My = df_member["My"].abs()
421         Mz = df_member["Mz"].abs()
422
423         # Design normal force
424         N_Rd = A*f_y
425         n = N/N_Rd
426
427         # Fraction to be used in Utilization Ratio
428         red = np.clip(1 - (N / N_Rd)**1.7, 0.1, 1.0)
429
430         # Utilization Ratio
431         df_member["Util_BNS"] = (My / (M_Rd * red))**2 + (Mz / (M_Rd * red))**2
432

```

Post_Process.py

```
433         return df_member["Util_BNS"]
434
435     Util_BNS_col = bnsFun(self.df_col, "R1", "R0", f_y) # Column
436     Util_BNS_brace = bnsFun(self.df_brace, "R3", "R2", f_y_brace) # Brace
437
438     return Util_BNS_col, Util_BNS_brace
439
440 # Buckling Resistance [6.3.1]
441 def Util_BR(self):
442
443     # Open and Read Eigenvalue
444     with open("Ansout/Eigenvalue1.txt") as f:
445         eigenvalues = [float(line.strip()) for line in f if line.strip()]
446         a_cr = next(v for v in eigenvalues if v > 0)
447
448     # Import Misc
449     f_y = self.f_y
450     f_y_brace = self.f_y_brace
451
452     def bucklingResFun(df_member, Ro, Ri, f_y):
453         df = df_member.copy()
454         util = np.zeros(len(df), dtype=float)
455
456         A = np.pi * (df[Ro]**2 - df[Ri]**2)
457
458         N_raw = df["NF"].to_numpy(dtype=float)
459         N_comp = np.maximum(-N_raw, 0.0)
460
461         active = N_comp > 0.0
462
463         if np.any(active):
464             a_imp = 0.49 # Table 6.1 CHS Profile Buckling Curve C Cold Roll
465             N_cr = a_cr * N_comp[active]
466
467             # P. 57
468             slen = np.sqrt((A[active] * f_y) / N_cr)
469             Phi = 0.5 * (1 + a_imp * (slen - 0.2) + slen**2)
470             Chi = 1.0 / (Phi + np.sqrt(Phi**2 - slen**2))
471             N_bRd = Chi * A[active] * f_y
472
473             util[active] = N_comp[active] / N_bRd
474
475         return pd.Series(util, index=df.index, name="Util_BR")
476
477     Util_BR_col = bucklingResFun(self.df_col, "R1", "R0", f_y) # Column
478     Util_BR_brace = bucklingResFun(self.df_brace, "R3", "R2", f_y_brace) # Brace
479
480     return Util_BR_col, Util_BR_brace
481
482 # Interaction Force [6.3.3]
483 def Util_IN(self):
484
485     # Imperfection Factor
486     a_imp = 0.49 # Table 6.1 CHS Profile Buckling Curve C Cold Roll
487
488     # Import Misc
489     f_y = self.f_y
490     f_y_brace = self.f_y_brace
491
492     # Open and Read Eigenvalue
493     with open("Ansout/Eigenvalue1.txt") as f:
494         eigenvalues = [float(line.strip()) for line in f if line.strip()]
```


Post_Process.py

```
495     a_cr = next(v for v in eigenvalues if v > 0)
496
497 def interaction(df_member, Ro, Ri, f_y):
498     df = df_member.copy().reset_index(drop=True)
499     util = np.zeros(len(df), dtype=float)
500
501     # Geometry (row-wise)
502     Do = 2 * df[Ro]
503     Di = 2 * df[Ri]
504     A = math.pi * (df[Ro]**2 - df[Ri]**2)
505
506     # Plastic and elastic section modulus for CHS
507     W_el_y = (np.pi * (Do**4 - Di**4)) / (32 * Do)
508     W_el_z = W_el_y
509
510     W_pl_y = (Do**3 - Di**3) / 6
511     W_pl_z = W_pl_y
512     # Eurocode shape factors Annex A EN 1993-1-1 6.3.3; w = W_pl/W_el capped at 1.5
513     w_y = np.minimum(W_pl_y / W_el_y, 1.5)
514     w_z = np.minimum(W_pl_z / W_el_z, 1.5)
515
516     N_raw = df["NF"].to_numpy(dtype=float)
517     N_comp = np.maximum(-N_raw, 0.0)
518
519     # Loop per structural member
520     for member, g in df.groupby("Member"):
521         idx = g.index.to_numpy()
522
523         N_c = N_comp[idx]
524         active = N_c > 0.0
525         if not np.any(active):
526             continue
527
528         # ---- END MOMENTS (per member) -----
529         My_start, My_end = g["My"].iloc[0], g["My"].iloc[-1]
530         M1y, M2y = (My_start, My_end) if abs(My_start) >= abs(My_end) else (My_end, My_start)
531         Psi_y = 0.0 if abs(M1y) < 1e-12 else np.clip(M2y / M1y, -1.0, 1.0)
532
533         Mz_start, Mz_end = g["Mz"].iloc[0], g["Mz"].iloc[-1]
534         M1z, M2z = (Mz_start, Mz_end) if abs(Mz_start) >= abs(Mz_end) else (Mz_end, Mz_start)
535         Psi_z = 0.0 if abs(M1z) < 1e-12 else np.clip(M2z / M1z, -1.0, 1.0)
536
537         # ---- BUCKLING / INTERACTION ----- (Calculations per element)
538         N_cr = a_cr * N_c[active]
539
540         # P. 57
541         slen = np.sqrt(A[idx][active] * f_y / N_cr)
542         Phi = 0.5 * (1 + a_imp * (slen - 0.2) + slen**2)
543         Chi = 1.0 / (Phi + np.sqrt(Phi**2 - slen**2))
544
545
546         # Annex A method 1 table A.1 (p. 76)
547         mu = (1 - N_c[active] / N_cr) / (1 - Chi * N_c[active] / N_cr)
548         # Annex A method 1 table A.2 (p. 78)
549         Cmy = 0.79 + 0.21 * Psi_y + 0.36 * (Psi_y - 0.33) * N_c[active] / N_cr
550         Cmz = 0.79 + 0.21 * Psi_z + 0.36 * (Psi_z - 0.33) * N_c[active] / N_cr
551         CmLT = 1.0
552
553         # Design Normal Force Resistance
554         N_Rk = A[idx][active] * f_y
555         # Auxiliary variable for interaction formula Annex A Table A.1 (p. 76)
556         npl = N_c[active]/N_Rk
```

Post_Process.py

```

557         # Auxiliary variables for interaction formula Annex A Table A.1 (p. 76)
558         wy = np.asarray(w_y, dtype=float)[idx][active]
559         wz = np.asarray(w_z, dtype=float)[idx][active]
560         # Auxiliary variables for interaction formula Annex A Table A.1 (p. 76)
561         Wel_over_Wpl_y = np.asarray(W_el_y, dtype=float)[idx][active] / np.asarray(W_pl_y, dtype=floa
    ][active]
562         Wel_over_Wpl_z = np.asarray(W_el_z, dtype=float)[idx][active] / np.asarray(W_pl_z, dtype=floa
    ][active]
563         # Interaction formula
564         Cyy = np.maximum(1+(wy-1)*((2-(1.6/wy) *Cmy**2*slen-1.6/wy *Cmy**2*slen**2)*npl),Wel_over_Wpl
565         Czy = np.maximum(1+(wz-1)*((2-14* (Cmz**2*slen**2/wz**5))*npl),0.6*np.sqrt(wz/wy)*(Wel_over_W
566
567         # Annex A method 1 table A.1 (p. 76)
568         k_yy = Cmy * CmLT * (mu / (1 - N_c[active] / N_cr)) / Cyy
569         k_yz = Cmz * CmLT * (mu / (1 - N_c[active] / N_cr)) / Czy *0.6*np.sqrt(wz/wy)
570
571         M_Rk = (math.pi * (Do[idx][active]**4 - Di[idx][active]**4)) / (32 * Do[idx][active]) * f_y
572
573         My = df.loc[idx, "My"].abs().to_numpy()[active]
574         Mz = df.loc[idx, "Mz"].abs().to_numpy()[active]
575
576         util_idx = idx[active]
577         util[util_idx] = (
578             N_c[active] / (Chi * N_Rk)
579             + k_yy * My / M_Rk
580             + k_yz * Mz / M_Rk
581         )
582
583         return pd.Series(util, index=df.index, name="Util_IN")
584
585         Util_IN_col = interaction(self.df_col, "R1", "R0", f_y) # Column
586         Util_IN_brace = interaction(self.df_brace, "R3", "R2", f_y_brace) # Brace
587
588         return Util_IN_col, Util_IN_brace
589
590     # Brace Step [Machine Design]
591     # Brace-Step
592     def Util_BS(self):
593
594         brace_sections = self.brace_horiz_d1_t1 if self.brace_split else self.brace_d1_t1
595
596         # Import Misc
597         f_y_brace = self.f_y_brace
598         E_mod = self.E_mod
599
600         # Force and Length
601         P = 150 * 9.82          # [N] 150 kg from standard
602         L = _brace_span_mm(self.var, self.Misc) # [mm] first horizontal brace (same for all)
603         M = 1/8 * P * L          # Max Moment [Nmm]
604
605         # Calculate utilization for all horizontal brace sections
606         util_values_sig = []
607         util_values_defl = []
608
609         for d1, t1 in brace_sections:
610             # Moment of Inertia and Area
611             I = np.pi / 64 * (d1**4 - (d1 - 2 * t1)**4) # [mm^4]
612             A = np.pi / 4 * (d1**2 - (d1 - 2 * t1)**2) # [mm^2]
613
614             # Sectional Modulus
615             W = I / (d1 / 2)                                # [mm^3]
616

```

Post_Process.py

```

617         # Bending Stress
618         sig_b = M / W                                # [N/mm^2]
619
620         # Vertical Force
621         V = P / 2                                    # [N]
622
623         # Average and Max Shear Stress
624         tau_avg = V / A                              # [N/mm^2]
625         tau_max = 2 * tau_avg                        # [N/mm^2]
626
627         # Von Mises
628         sig_vm = np.sqrt(sig_b**2 + 3 * tau_max**2) # [N/mm^2]
629
630         util_values_sig.append(sig_vm / f_y_brace)    # [Na] Brace
631
632         # Deflection (u_max = (1/192) * ((P * L^3) / (E * I))) - Maskinståbi
633         u_max = (P * L**3) / (192 * E_mod * I)       # [mm]
634
635         util_values_defl.append(u_max / (L / 200))    # [Na] Brace Max deflection from standard
636         # Return as Series to allow extract_max to work with multiple values
637         return pd.Series(util_values_sig, name="Util_BS_sig"), pd.Series(util_values_defl, name="Util_BS_defl")
638
639     def Class_2(self):
640         # For this function we use it for a constraint, such that we always have a Class 2 section or below a
ng to
641         # Eurocode 3 Section Table 5.2
642         # And for the optimization scheme we need to implement the form:
643         # c(x) >= 0 "Inequality Constraint"
644         # Therefore we get:
645         # 70*235/f_y - d/t >= 0 for each tube size
646
647         f_y = self.f_y
648         f_y_brace = self.f_y_brace
649
650         # Chat code for calculating class 2 for all elements
651         # --- Column -----
652         col_constraints = [
653             70 * (235 / f_y) - (d / t)
654             for d, t in self.column_d0_t0
655         ]
656
657         # --- Braces -----
658         brace_constraints = []
659
660         if self.brace_split:
661             # Horizontal braces
662             brace_constraints.extend(
663                 70 * (235 / f_y_brace) - (d / t)
664                 for d, t in self.brace_horiz_d1_t1
665             )
666
667             # Cross braces
668             brace_constraints.extend(
669                 70 * (235 / f_y_brace) - (d / t)
670                 for d, t in self.brace_cross_d1_t1
671             )
672         else:
673             # Same size used for all braces
674             brace_constraints.extend(
675                 70 * (235 / f_y_brace) - (d / t)
676                 for d, t in self.brace_d1_t1
677             )

```

```
678
679     return np.asarray(col_constraints + brace_constraints, dtype=float)
680
681
682 def Eigenvalue_1(self):
683
684     # We implement a constrain, such that the first positive eigenvalue is greater than or equal to 4.0
685     # And for the optimization scheme we need to implement the form:
686     #  $c(x) \geq 0$  "Inequality Constraint"
687     # Therefore we get:
688     #  $a_{cr} \geq 4.0$ 
689
690     # Open and Read Eigenvalue
691     with open("Ansout/Eigenvalue1.txt") as f:
692         eigenvalues = [float(line.strip()) for line in f if line.strip()]
693         a_cr = next(v for v in eigenvalues if v > 0)
694
695     # Return Constraint Value
696     # SLSQP inequality form:  $c(x) \geq 0$ 
697     return a_cr - 4.0
```